Integration Layer Design: 3D Sales Workspace

Backend Assessment, Part A (architecture and design) · Candidate: [Your Name] · 20 October 2026

0. Summary

The workspace puts five enterprise tools beside one 3D electrical room. This document describes the backend layer between the React frontend and those tools. It cannot change any of them, and it must keep the sidebar useful within 1 s at p95 for up to 600 concurrent sessions, while respecting rate limits of 20 and 5 requests per second, per-user permissions and nightly outages.

- **Shape:** a **modular monolith backend-for-frontend (BFF)** with **one adapter per enterprise system**, plus a **worker process** for slow and background work and an **assistant service** beside it. It is not microservices: the bottlenecks are external rate limits, which splitting our code would not raise.
- **Stack:** Node.js + TypeScript + Express, **PostgreSQL** for durable, relational data, **Redis** for cache, global rate limits, queues (BullMQ) and pub/sub, all on Azure Container Apps inside the customer's tenant.
- **Panel contract:** every sidebar section returns its own status (`ok`, `stale`, `pending`, `unavailable`, `not_permitted`). Slow results arrive later over **Server-Sent Events (SSE)**.
- **Identity:** OIDC in the browser, then **on-behalf-of (OBO) token exchange** to systems that hold per-user permissions (sizing, documents, host), and a **service principal** for the catalogue and the ERP. Price and document visibility are enforced in **one server-side entitlement layer** that every path uses, including the AI assistant.
- **Consistency:** a placed item has one stable `item_id`. A re-size changes its variant, 3D model, BOM line and price in **one Postgres transaction**, and an **outbox** carries the change to the host.
- **AI:** a private LLM whose tools are thin wrappers over the same BFF services, so it can never see more than the user. Writes require a confirmation that the model cannot give itself.

Contents

1. Architecture diagram (A1)
2. Architecture explanation
3. Technology choices
4. Protocols and trust boundaries
5. Identity and entitlements
6. Linking the 3D scene to data
7. Latency, limits, outages and versions
8. Catalogue sync, search and 3D mapping
9. AI assistant, security and operations
10. Key sequences (1 to 3)
11. Rejected alternatives
12. Assumptions
13. What Part B implements

1. Architecture diagram (A1)

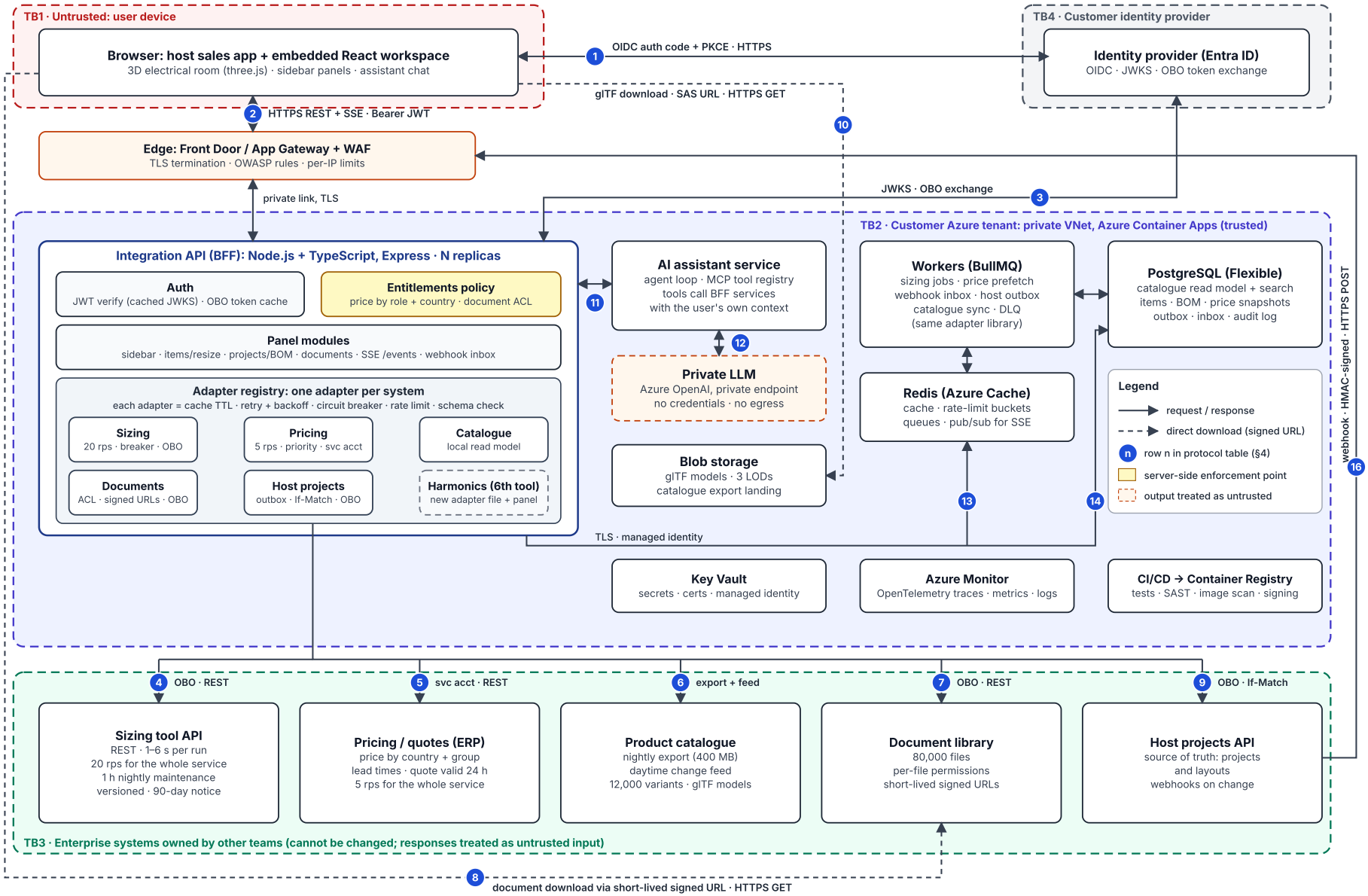


Figure 1. Components, data flows, protocols and trust boundaries (TB1 to TB4). Numbered badges refer to rows in the protocol table in §4. Dashed boxes: the 6th tool (future adapter) and the private LLM, whose output is treated as untrusted.

2. Architecture explanation

2.1 Overall shape: modular monolith BFF

One deployable **Integration API** serves the frontend. Inside, code is split into modules with strict import rules: `core` (auth, entitlements, resilience helpers), `modules` (one per panel or use case) and `adapters` (one per enterprise system). The same codebase runs as two more processes: **workers** (BullMQ consumers for sizing jobs, price prefetch, webhook inbox, host outbox, catalogue sync) and the **assistant service**.

- **Why a BFF and not a generic gateway:** the sidebar needs aggregation, a per-section status and entitlement filtering. A gateway product (for example Azure API Management) can route and throttle but cannot hold that logic, so it sits in front only as the edge, if at all.
- **Why one deployable:** 600 concurrent sessions is modest. In-process fan-out is faster than extra network hops, a single entitlement layer is easier to prove correct, and every extra service adds endpoints to the security review and penetration test.
- **Why workers are separate:** sizing runs (1 to 6 s), the 400 MB catalogue import and retries to the host have a different load profile and must never slow down interactive requests.
- **Escape hatch:** adapters are isolated behind an interface, so any one can be extracted into its own service if scale or team ownership demands it (see §9.4).

2.2 How a sidebar panel talks to the backend (panel contract)

The frontend makes **one aggregate call** on selection for first paint, and can refresh a **single panel** independently. Both return the same envelope per section:

```
GET /api/items/{itemId}/sidebar            → all panels for first paint (≤ 1 s)
GET /api/items/{itemId}/panels/{panelId}   → one panel (retry / refresh)
GET /api/events                            → SSE stream: panel.updated, item.updated, project.changed

{ "itemId": "DRV-03", "variantCode": "ACS-R7-75-400", "itemVersion": 7,
  "sections": {
    "catalogue": { "status": "ok", "data": {...}, "fetchedAt": "..." },
    "sizing":    { "status": "stale", "data": {...}, "reason": "sizing_unavailable" },
    "pricing":   { "status": "pending", "data": null, "jobId": "pr_9f2" },
    "documents": { "status": "ok", "data": [...] },
    "harmonics": { "status": "not_permitted", "data": null } } }
```

Status	Meaning and UI behaviour
`ok`	Fresh data within its TTL.
`stale`	Cached data, shown with its age, because the source failed, timed out, is in maintenance or its circuit is open.
`pending`	Work is queued (slow sizing, rate-limited price). The panel shows a placeholder; the result arrives on the SSE stream.
`unavailable`	The source failed and nothing cached exists. The panel shows a clear message; other panels are unaffected.
`not_permitted`	The caller is not entitled. No data and no hint of the value is sent.

2.3 Adapter contract and adding a 6th tool

Every enterprise system sits behind the same interface. Resilience is configuration, not code copied into each adapter:

```
interface Adapter<Q, R> {
  id: "sizing" | "pricing" | "catalogue" | "documents" | "host" | string;
  policy: { cacheTtlMs; staleTtlMs; timeoutMs; retries; rateLimit?: { rps; priorityLanes };
            breaker?: { failureRate; windowMs; coolDownMs }; tokenMode: "obo" | "service" };
  schema: ZodSchema<R>;                      // validates every vendor response
  fetch(query: Q, ctx: CallerContext): Promise<R>;
}
interface PanelProvider { panelId; requiredEntitlement?; load(item, ctx): Promise<SectionResult> }
```

Adding a harmonics calculator means adding one `adapters/harmonics/` folder (client, schema, policy), one `PanelProvider` and one registry line. The sidebar service iterates over registered providers, so no other adapter or panel changes. The frontend renders panels from a `/api/panels` manifest, so the new panel appears without a new endpoint.

3. Technology choices

Component	Choice	Reason
Runtime and API	Node.js 20 LTS, TypeScript (strict), Express	Non-blocking I/O suits fan-out to many slow systems. Types make the panel and adapter contracts explicit. Express is small and well understood, which matters for security review and for changing code live.
Durable data	Azure Database for PostgreSQL, Flexible Server (zone-redundant HA)	The data is relational: project → item → variant → BOM line → price snapshot; product model → variant → 3D model → documents. A re-size needs a multi-row transaction. JSONB covers variable technical attributes; full-text search plus `pg_trgm` covers search over 12,000 variants without a separate search engine.
Cache, rate limits, queues, pub/sub	Azure Cache for Redis (with persistence) + BullMQ	One component gives a shared TTL cache, **global** token buckets across replicas (the 20 and 5 rps limits are per service, not per replica), priority queues with job-ID dedupe, retries and a dead-letter queue, and pub/sub to route SSE events to the replica holding the connection.
Push to browser	Server-Sent Events	Push is one-way (server → browser). SSE works over plain HTTPS through the WAF, reuses the bearer token, and reconnects automatically. Requests from the browser stay REST.
Validation	zod	Validates requests, configuration and **every vendor response**, which is how a vendor API change is detected (§7.5).
Auth	jose (JWT/JWKS), MSAL for OBO	Standards-based verification of signature, issuer, audience and expiry; Entra ID supports the OAuth 2.0 on-behalf-of flow.
3D assets	Azure Blob Storage (+ CDN/Front Door cache)	glTF files of 2 to 40 MB should not flow through the API. Content-hashed paths are immutable and cacheable; browsers fetch with short-lived read-only SAS URLs.
AI	Private LLM (Azure OpenAI on a private endpoint, or a self-hosted model in the tenant); MCP for tool definitions	No customer data leaves the tenant. MCP gives a standard tool schema, and the tools are implemented by the BFF's own services.
Hosting	Azure Container Apps in a VNet; Front Door or Application Gateway with WAF	Containers as required, autoscaling by HTTP and queue depth, internal ingress for workers and the assistant, managed identity for every component.

Secrets	Azure Key Vault + managed identity	No secrets in images, repos or environment files.
Observability	pino (JSON logs), OpenTelemetry → Azure Monitor / Application Insights	One trace per user action across adapters, with per-adapter metrics and alerts.
CI/CD	GitHub Actions or Azure DevOps → Azure Container Registry	Lint, typecheck, unit and contract tests, SAST, dependency and image scanning, SBOM, signed images, staged rollout.

4. Protocols and trust boundaries (every connection)

Trust boundaries: TB1 user device (untrusted) → edge; TB2 edge → customer tenant (private VNet); TB3 tenant → enterprise systems owned by other teams (authenticated both ways; responses validated as untrusted input); TB4 tenant ↔ identity provider. Inside TB2, the LLM is a lower-trust zone: it holds no credentials and its output is never executed without checks.

#	Connection	Protocol	Credential	Notes
1	Browser ↔ Identity provider	OIDC authorization code + PKCE over HTTPS	User sign-in (SSO shared with the host app)	Access token audience = Integration API; lifetime 60 min or less.
2	Browser → WAF → Integration API	HTTPS (TLS 1.2+), REST/JSON; SSE (`text/event-stream`)	Bearer access token	WAF: OWASP rule set, per-IP limits, only `/api/*`. Per-user limits in the API. CORS restricted to the host origin.
3	Integration API → Identity provider	HTTPS: JWKS fetch (cached); OAuth 2.0 token exchange (OBO); client credentials	API's managed identity / certificate	OBO tokens cached per user and audience until shortly before expiry.
4	Adapters → Sizing API	HTTPS REST/JSON, pinned API version	OBO user token	Global 20 rps token bucket; circuit breaker; known maintenance window.
5	Adapters → Pricing ERP	HTTPS REST (or OData)	Service principal	Global 5 rps with priority lanes. Entitlement checked by us **before** the call.
6	Workers → Catalogue	Nightly export file via HTTPS/SFTP to Blob; change feed via HTTPS (poll or push)	Service principal	No user data. Imported into the Postgres read model.
7	Adapters → Document library	HTTPS REST	OBO user token	The library applies per-file ACLs; we filter again and request signed URLs.
8	Browser → Document library	HTTPS GET on a signed URL	Signed URL (≤ 5 min, one file)	File bytes never pass through our API.
9	Adapters/workers → Host projects API	HTTPS REST, `If-Match` / ETag	OBO user token (service principal for outbox retries, with acting-user header if supported)	Optimistic concurrency prevents lost updates.

10	Browser → Blob storage (glTF)	HTTPS GET via CDN	User-delegation SAS, read-only, prefix-scoped, ≤ 60 min	Content-hashed, immutable files; LODs loaded progressively.
11	Integration API ↔ Assistant	Internal HTTPS (Container Apps internal ingress); answers streamed to the browser via SSE	User's token forwarded; service identity	Assistant tools call BFF services with the user's caller context.
12	Assistant ↔ Private LLM	HTTPS over private endpoint	Managed identity	No public network access; no outbound internet from the LLM.
13	API/workers ↔ Redis	TLS (port 6380)	Entra auth / key from Key Vault	Private endpoint.
14	API/workers ↔ PostgreSQL	TLS (port 5432)	Managed identity (Entra auth)	Private endpoint; least-privilege roles (API cannot alter schema).
15	All → Key Vault, Azure Monitor	HTTPS; OTLP/HTTPS	Managed identity	Shown as boxes in Figure 1.
16	Host → WAF → API (webhook)	HTTPS POST	HMAC signature + timestamp	Rejected outside a 5-minute window; deduplicated by event ID; queued to the inbox and acknowledged in under 200 ms.

5. Identity and entitlements

5.1 How the user's token travels

The workspace is embedded in the host application and shares its SSO session. The single-page app obtains an access token for the Integration API (row 1). The API validates it on every request: signature against cached JWKS, issuer, audience, expiry and not-before. It then builds a **caller context** (`userId, role, country, customerGroup, tokenRef`). The raw token never reaches the browser's other calls or the LLM.

Downstream	Token type	When and why
Sizing, documents, host	Delegated (OBO) user token	These systems hold per-user permissions or audit by user. OBO keeps their own checks in force and their audit trails accurate.
Pricing ERP	Service principal	Assumed: the ERP does not accept end-user tokens. Our entitlement layer is the gate. If the ERP supports OBO, use it, and keep our check as a second layer.
Catalogue	Service principal	Bulk, non-personal data read by background jobs.
Background retries (outbox)	Service principal, acting-user recorded	The user's token may have expired by the time a retry runs.

5.2 Server-side enforcement

- **One policy module** (`core/entitlements`) answers `canSeePrice(ctx)`, `canSeeDocument(ctx, doc)` and `canEditProject(ctx, project)`. Policy is data (role → permissions, per-country rules), so it can change without code.
- **Price** (and lead time, see assumptions) is removed **before** serialisation for `channel_partner` and other restricted roles. Pricing is not even called for them. Prices are always for the caller's country and customer group.

- **Documents:** the list comes from the library under the user's OBO token and is filtered again against our cached ACL. Signed URLs are issued per user and per file, never cached across users.
- **Every path** goes through it: sidebar, BOM, SSE events (events are filtered per subscriber), exports and assistant tools. Tests assert that no price field appears anywhere in a channel partner's JSON.
- **Object-level checks:** every `itemId` and `projectId` is checked against project membership, which prevents insecure direct object references.

6. Linking the 3D scene to data

A placed item has a stable `item_id` created when it is placed. Everything else is a reference that can change:

```
item (item_id, project_id, kind, variant_code, model_ref, position, motor_input, sizing_ref, version)
  ├── variant_code → catalogue_variant (code, model_id, attributes jsonb, revision)
  │                    └── model_id → model_asset (lod0_path, lod1_path, lod2_path, content_hash)
  ├── sizing_ref   → sizing_result (input_hash, api_version, variant_code, load_pct)
  ├── bom_line     (item_id UNIQUE, variant_code, quantity, options)
  ├── price        → price_snapshot (variant, country, group, amount, lead_time, valid_until)
  └── documents    → document_link (variant_code, doc_id)   (ACL applied at read time)
```

- **Re-size consistency:** when sizing recommends a new variant, one transaction updates `variant_code`, `model_ref`, `sizing_ref`, the BOM line and increments `version`, and inserts an outbox row for the host. Prices and documents are derived from the variant, so they cannot drift.
- **Concurrent edits:** writes carry `If-Match: version`; a mismatch returns 409 and the client reloads the item.
- **Source of truth:** the host owns the layout; Postgres is our working copy. Changes go to the host through the outbox; host changes come back through webhooks (inbox), applied only if their version is newer.

7. Latency, limits, outages and versions

7.1 Sidebar latency budget (1 s at p95)

- Selection is highlighted in 3D immediately on the client; no network is needed.
- The API loads the item (~5 ms), then calls all panel providers **in parallel**, each with its own deadline (≈ 800 ms), and responds by ~900 ms with whatever has arrived. Late sections become `pending` and are pushed over SSE.
- The **sizing panel shows the item's stored sizing result**; the sizing tool is only called when inputs change (re-size). This keeps the slowest system off the selection path.
- Opening a project triggers a low-priority **prefetch** of prices and document lists for all its items, so later clicks are cache hits.

7.2 Caching

Data	Key	Fresh / stale TTL	Scope	Notes
Catalogue	Postgres read model; Redis `cat:{variant}`	Read model always; Redis 1 h	Shared	Invalidated by change-feed revision.
3D models	content hash	Immutable	Shared (CDN + browser)	New model = new path.
Sizing result	`size:{hash(inputs, apiVersion)}`	15 min / 24 h; stored per item in Postgres	Shared	Assumes output depends only on inputs.
Price	`price:{variant}:{country}:{group}`	10 min / until quote `valid_until` (24 h)	Shared per country + group	Entitlement checked **before** reading the cache. Never served after `valid_until`.

Document list	`docs:{variant}` + per-user ACL	30 min list; 5 min ACL decision	List shared, ACL per user	Signed URLs are per user and never cached.
OBO tokens	`obo:{user}:{audience}`	Until expiry − 5 min	Per user	Encrypted at rest in Redis.
Project	`proj:{id}:{version}`	Until webhook	Per project	Invalidated by host webhook.

7.3 Rate limits, queues and priorities

- **Global token buckets in Redis** (atomic Lua script): sizing 20 rps, pricing 5 rps, shared by all replicas. Limits are configuration and can change at runtime.
- **Priority lanes** (BullMQ): P1 user-initiated re-size; P2 sidebar cache miss; P3 project prefetch and BOM refresh; P4 assistant bulk queries. When the bucket is empty, a P2 request waits only within its deadline, then returns `stale` or `pending`.
- **Deduplication:** the job ID is the cache key, so 50 users asking for the same price or sizing cause one call.
- **Capacity check:** 600 sessions with one selection per ~20 s is about 30 sidebar requests per second. With prefetch and shared price caching, the expected miss rate is well under 5 rps. If pricing drops to 1 rps, the levers are longer fresh TTLs (quotes are valid for 24 h), prefetch on project open, and showing stale prices with their age.
- **Retries:** at most 2, exponential backoff with jitter, only for timeouts, 5xx and 429 (honouring `Retry-After`), and only if they fit in the caller's deadline. Failed background jobs go to a dead-letter queue with alerting.

7.4 Outages and the maintenance window

- **Circuit breaker per adapter** (mandatory for sizing): opens on a failure-rate threshold in a rolling window, half-opens after a cool-down with one probe request.
- **Nightly sizing maintenance**: the window is configured, so the breaker is opened in advance instead of waiting for failures. The sizing panel shows the stored result as `stale` with a banner; re-size is disabled with a clear message. Requests are not queued for an hour.
- Behaviour for every system is in §9.3.

7.5 Detecting vendor API changes before users do

- Every adapter pins the API version and validates responses with a zod schema. A schema mismatch is a distinct error type with its own metric and alert, and the section degrades to `stale`.
- **Contract tests** run nightly and in CI against each vendor's sandbox (or recorded examples), so a breaking change fails a pipeline, not a user.
- Version notices (90 days for sizing) are tracked as backlog items. The adapter supports old and new versions behind a flag, with shadow calls comparing results before switching.

8. Catalogue sync, search and 3D mapping

- **Nightly import:** the 400 MB export lands in Blob. A worker streams and parses it (never fully in memory), loads it into staging tables, validates row counts and checksums, then swaps it in within one transaction. Users never see a half-loaded catalogue, and a failed import keeps yesterday's data.
- **Change feed:** events are queued and applied in revision order with upserts (`WHERE incoming.revision > current.revision`), so they are idempotent and an old update never overwrites a newer one. The next nightly import reconciles any gap.
- **Search:** Postgres full-text search over names and descriptions plus trigram indexes on codes (for example "R7", "ACS880"), with indexed columns for facets such as power, voltage, frame and IP rating. 12,000 rows fit comfortably; no separate search engine.
- **3D mapping:** `variant → model_id → model_asset` with three LOD paths. Assets are stored once per content hash in Blob. The browser loads the lowest detail level first, then upgrades, so a 40 MB model never blocks

interaction.

9. AI assistant, security and operations

9.1 AI assistant

- **Tools** (MCP definitions): `get_project_layout`, `get_item_details`, `get_bom_cost`, `get_lead_times`, `search_catalogue`, `list_documents`, and the write tool `resize_item`. Each tool is a thin wrapper over the same BFF services and adapters, called with the **user's caller context**. A channel partner's `get_bom_cost` returns `not_permitted`, so the model never receives a price it could leak.
- **Write confirmation:** `resize_item` returns a *proposal* (old and new variant, BOM and price delta) and a signed, single-use confirmation token bound to user, item, arguments and a 5-minute expiry. The UI shows a confirmation dialog; only the user's click calls the API with that token. The model has no way to confirm on the user's behalf.
- **Prompt-injection defences** (catalogue text and documents are untrusted): (1) privilege separation: the model has at most the user's permissions, so injection cannot escalate; (2) human confirmation for every write, outside the model; (3) no exfiltration channel: no web or URL tools, no outbound network, and the chat UI does not auto-load images or links from model output; (4) untrusted text is passed in clearly delimited data fields and never concatenated into instructions; (5) limits on tool calls per turn and argument validation against schemas.
- **Audit** (Postgres, append-only, 1-year retention in the tenant): user, question, each tool call with arguments, entitlement decisions (including denials), proposals, confirmations and the final answer.

9.2 Security and operations

- **Secrets:** Key Vault and managed identities only; secret scanning in CI.
- **Logging:** structured JSON with request and trace IDs; tokens, signed URLs and prices are redacted.
- **Observability:** per adapter: latency, error rate, breaker state, queue depth, cache hit ratio, schema violations. Product SLO: sidebar p95 < 1 s and the share of panels not `ok`. Alerts on breach.
- **CI/CD:** lint, typecheck, unit, integration and contract tests → SAST (CodeQL), dependency audit, container scan (Trivy), SBOM → signed image to ACR → staging with smoke tests → security review and penetration-test gate → production as a new Container Apps revision with traffic splitting and instant rollback.

Likely penetration-test finding	Mitigation
Insecure direct object reference on `itemId`/`projectId`	Project membership check on every request; tests per endpoint.
Price leakage via aggregates, errors, cache, SSE or the assistant	Single entitlement layer, per-subscriber SSE filtering, tests that scan JSON for price fields.
JWT weaknesses (algorithm confusion, missing audience check, long lifetimes)	Fixed algorithm list, issuer and audience checks, short token lifetimes, clock-skew limit.
Webhook spoofing or replay	HMAC signature, timestamp window, event-ID dedupe.
SSRF through model or document URLs from vendor data	The API never fetches URLs from data; allow-listed outbound hosts only.
Over-long signed URLs; CORS too broad; verbose errors	Short lifetimes, host-origin-only CORS, generic error bodies with a correlation ID.
Missing rate limits on our own API; vulnerable dependencies	Per-user and per-IP limits; automated dependency updates and scanning.

9.3 Behaviour when each system fails

Failure	Behaviour
Sizing down / maintenance	Breaker open; stored result shown as `stale`; re-size disabled with a message.
Pricing down or throttled	Cached price as `stale` with age; after `valid_until` it becomes `unavailable` ("price on request"); BOM total marked incomplete.
Catalogue source down	No user impact (local read model); sync alert; freshness banner if data is over 36 h old.
Document library down	Cached list shown as `stale`; downloads unavailable.
Host down	Reads from our working copy; saves go to the outbox and retry; UI shows "saved, syncing".
Identity provider down	Existing tokens work until expiry; cached OBO tokens continue; new sign-ins fail.
LLM down	Assistant panel `unavailable`; nothing else affected.
Redis down	Direct calls with an in-process limiter at a conservative share (limit ÷ replicas); queues pause; SSE clients fall back to polling.
PostgreSQL down	Zone-redundant failover; meanwhile read-only from Redis cache and writes return 503.

9.4 What breaks first as usage grows

- **5,000 users (~600 concurrent):** the design holds. First pressure: the 5 rps pricing limit on a cold cache (Monday morning, new price list). Mitigation: scheduled prefetch of prices for active projects and a longer fresh TTL.
- **50,000 users (~6,000 concurrent):** external limits become hard blockers. We need a negotiated bulk price-list feed replicated locally, and higher sizing quotas or a local sizing engine. Next: LLM throughput (GPU capacity and cost), SSE connections (move to Azure Web PubSub), Redis (cluster mode), CDN egress for 3D models. At this point extracting the assistant and the busiest adapters into separately scaled services becomes worthwhile.

10. Key sequences

Sequence 1: user clicks a drive; the sizing tool is down

1. User clicks DRV-03. The 3D view highlights it immediately (client only).
2. Browser: `GET /api/items/DRV-03/sidebar` with Bearer token → WAF → API.
3. API validates the token and builds the caller context (sales engineer, DE, group G1), checks project membership and loads the item from Postgres.
4. Panel providers run in parallel, each with an ~800 ms deadline:
   - **Catalogue:** read model / Redis hit → `ok`.
   - **Sizing:** the item's stored result is current → `ok`. If the inputs had changed, a live call is needed, but the breaker is open (sizing down), so no call is made: the last result is returned as `stale` with "sizing unavailable since 13:40", or `unavailable` if none exists.
   - **Pricing:** entitlement allows → Redis hit for R7/DE/G1 → `ok`. On a miss: token bucket → ERP call within the deadline, or a P2 job → `pending`.
   - **Documents:** cached OBO token → list (cache or library) filtered by ACL → `ok`.
5. API responds within ~900 ms with each section and its status. The UI renders each panel independently.
6. Any `pending` section arrives as a `panel.updated` SSE event, routed via Redis pub/sub to the replica holding this user's connection.

Sequence 2: load increases → re-size to a larger frame

1. User raises motor power from 75 to 90 kW. Browser: `POST /api/items/DRV-03/resize` with `Idempotency-Key` and `If-Match: 7`.
2. API checks token, edit permission, idempotency key and the breaker state (if open: 503 "sizing unavailable", nothing changes).
3. Sizing adapter: cache miss for `hash(inputs, apiVersion)` → P1 job (job ID = hash, so duplicates merge). The API waits up to ~1.5 s for it.
4. Worker takes a token from the 20 rps bucket, gets the OBO token, calls the sizing API (1 to 6 s), validates the response schema and stores the result.
5. If the result arrived in time, the API continues in the same request. Otherwise it returns `202 {status: pending, jobId}` and the steps below finish server-side, with the outcome pushed over SSE.
6. Recommended variant R8 differs from R7 → one transaction: item variant, `model_ref`, `sizing_ref`, BOM line, `version = 8`, outbox row for the host.
7. Price for R8 (P1, entitlement checked) → `ok` or `pending`.
8. Response or `item.updated` event: new variant, SAS URLs for the R8 model LODs, BOM change (−R7, +R8), price delta. The client swaps the 3D model (low LOD first), re-runs layout rules (a larger cabinet may now violate clearance and turns red) and updates the BOM and price panels.
9. Outbox worker updates the host project with `If-Match`; on conflict it reconciles and notifies the user. The host's webhook echo is ignored as already applied (same version).
10. If the variant did not change: only the sizing result is updated; no BOM change and no host write.

Sequence 3: "What will this layout cost, and which item has the longest lead time?"

1. Browser: `POST /api/assistant/messages` (answer streamed over SSE). The API passes the question and the caller context to the assistant service.
2. The assistant sends the question, tool definitions and project ID to the private LLM. The model calls `get_bom_cost(projectId)` and `get_lead_times(projectId)`.

3. Each tool calls the BFF project service with the **user's** context, through the same entitlement layer and adapters (pricing at P4, mostly cache hits from the project prefetch).
4. **Sales engineer:** tools return lines with prices (DE, group G1), totals, quote validity and lead times. The model answers, for example: "Estimated total €184,300 (prices valid until 21 Oct 14:02). Longest lead time: DRV-03 (frame R8), 12 weeks." Any `stale` or `pending` price is stated as such, never guessed.
5. **Channel partner:** tools return `{pricing: "not_permitted"}` with item counts only. The model never receives a price or lead time, so no prompt can extract one. It answers: "Pricing and lead times aren't available for your role. The layout has 6 items; your sales contact can provide a quote."
6. Question, tool calls, entitlement decisions and answer are written to the audit log.

11. Rejected alternatives

Alternative	Why rejected
Microservice per adapter	More network hops in a 1 s budget, cross-service coordination for global rate limits, more attack surface for every release review. No independent scaling need at 600 sessions. Adapters stay extractable.
MongoDB (reference stack)	The workspace data is relational, and a re-size is a multi-entity transaction. Postgres JSONB covers variable product attributes.
GraphQL gateway or federation	Good for flexible queries, but adds a schema layer, and partial-failure states are less explicit than a per-section status. The panel set is small and fixed.
Kafka or Azure Service Bus	Redis + BullMQ meets the throughput, priority and dedupe needs with one fewer component. Service Bus is the upgrade path if stronger delivery guarantees are needed.
Elasticsearch / Azure AI Search	12,000 variants fit Postgres full-text and trigram search. Another cluster would add cost and operations.
WebSockets	Push is one-way; SSE is simpler and works over plain HTTPS through the WAF.
Browser calling enterprise systems directly	Exposes tokens and vendor APIs, makes global rate limits impossible and leaves entitlements to the UI.
One service account for every system	Bypasses per-user permissions in sizing, documents and host, and loses user-level audit.
Queueing sizing requests during maintenance	An hour-long "pending" is worse than a clear "unavailable" with the last known result.

12. Assumptions

1. The identity provider is Microsoft Entra ID (OIDC) and supports the on-behalf-of flow; enterprise APIs trust its tokens.
2. The workspace is launched from the host application and shares its SSO session; the SPA obtains its own token for the Integration API.
3. The token (or a lookup from the host) provides role, country and customer group.
4. Sizing results depend only on the inputs and API version, so they can be cached and shared between users.
5. The ERP accepts a service principal, not user tokens; price entitlement is therefore enforced by our layer.
6. **Lead time is commercially sensitive and follows price visibility** (the mockup groups them in one entitled panel). This is a per-field policy setting, so it can be changed.
7. The host API supports ETag/`If-Match` and sends signed webhooks with event IDs and versions. If not, we poll and compare versions.

8. The catalogue export arrives as a file (HTTPS/SFTP) and the change feed carries monotonically increasing revision numbers.
9. The document library exposes per-file ACLs via API and issues signed URLs itself; we never proxy file bytes.
10. Peak usage is about one selection per user per 20 seconds (~30 sidebar requests per second at 600 sessions).
11. The sizing maintenance window is at a known, published time.
12. The private LLM supports tool (function) calling; all components run in one Azure region chosen by the customer.

13. What Part B implements

The submitted service is a deliberately small slice of this design: the Integration API with three adapters (sizing, catalogue, pricing) on shared resilience helpers (per-system cache TTL, retry with backoff, circuit breaker on sizing, Redis rate limiter), the sidebar, re-size and project BOM endpoints, JWT-based entitlement filtering, PostgreSQL and Redis via Docker Compose, mocks for the three systems, and tests. In Part B, sizing in the sidebar is a direct call with a deadline (`ok`, `stale` or `unavailable`); the queue, workers, SSE push, outbox, webhooks, catalogue sync and assistant are described here and listed as optional or future work in the README.